

finally, write_env, and unstoppable Sender Adaptors

Document #: P3284R1
Date: 2024-07-16
Project: Programming Language C++
Audience: LEWG Library Evolution
Reply-to: Eric Niebler
<eric.niebler@gmail.com>

Contents

- [1 Introduction](#)
- [2 Executive Summary](#)
- [3 Description](#)
 - [3.1 write_env](#)
 - [3.1.1 Example: write_env](#)
 - [3.2 unstoppable](#)
 - [3.3 finally](#)
 - [3.3.1 Example: finally](#)
- [4 Discussion](#)
 - [4.1 Should finally apply unstoppable by default to its second argument?](#)
 - [4.2 Is there a different design that better captures the “async RAII” intent?](#)
- [5 Proposed Wording](#)
- [6 References](#)

§ 1 Introduction

This paper proposes to add three new sender adaptor algorithms to the `std::execution` namespace, targeting C++26: `finally`, `write_env`, and `unstoppable`. These adaptors were originally proposed as part of [P3175R0] but have been split out into their own paper so that the higher priority items in P3175 can advance more quickly.

§ 2 Executive Summary

Below are the specific changes this paper proposes:

1. Add a new uncustomizable adaptor `write_env` for writing values into the receiver's execution environment. Use `write_env` in the implementation of the `on` algorithm and to simplify the specification of the `let_` algorithms.
2. Add an uncustomizable unstopable adaptor that is a trivial application of `write_env`: it sets the current stop token in the receiver's environment to a `never_stop_token`. `unstopable` is used in the re-specification of the `schedule_from` algorithm.
3. Generalize the specification for `schedule_from` to take two senders instead of a sender and a scheduler, name it `finally`, and make it uncustomizable. Specify the default implementation of `schedule_from(sch, snd)` as `finally(snd, unstopable(schedule(sch)))`.

§ 3 Description

[P3175R0] proposed some changes to the `std::execution::on` algorithm, the specification of which was made simpler by the addition of some additional adaptors. Those adaptors were general and useful in their own right, so P3175R0 suggested they be added to `std::execution` proper. The conservative approach was to make them exposition-only, and that is how things currently stand in the working draft.

The author still feels like those adaptors are worthy of standardization. This paper proposes adding them.

The adaptors in question are as follows:

§ 3.1 `write_env`

A receiver has an associated “execution environment”, which is an unstructured, queryable key/value store. It is used to pass implicit parameters from parent operations to their children. It is occasionally useful for a sender adaptor to explicitly mutate the key/value store so that child operations see different values for environment queries. The `write_env` sender adaptor is used for that purpose.

`write_env` is a customization point object, although it is not actually customizable. It accepts a sender `sndr` and an execution environment `env`, and it returns a new sender that stores `sndr` and `env`. When that sender is connected to a receiver `rcvr`, it returns the result of connecting `sndr` with a receiver that adapts `rcvr`. The environment of that adapted receiver is the result of joining `env` with `rcvr`'s environment. The two environments are joined such that, when the joined environment is queried, `env` is queried first, and if `env` doesn't have a value for that query, the result of `get_env(rcvr)` is queried.

§ 3.1.1 Example: `write_env`

One example of where `write_env` might be useful is to specify an allocator to be used by child operations. The code might look like this:

```
// Turn a query object and a value into a queryable environment
// (see [P3325R2]):
template <class Query, class Value>
struct prop {
    Query query;
    Value value;
    decltype(auto) query(Query) const noexcept { return (value); }
};

// Adapts a sender so that it can use the given allocator:
struct with_allocator_t {
    template <std::execution::sender Sndr, class Alloc>
    auto operator()(Sndr sndr, Alloc alloc) const {
        return std::execution::write_env(sndr, prop(std::get_allocator,
            alloc));
    }

    template <class Alloc>
    auto operator()(Alloc alloc) const {
        return std::execution::write_env(prop(std::get_allocator, alloc));
    }
};

constexpr with_allocator_t with_allocator{};
```

The `with_allocator` adaptor might be used to parameterize senders produced by a third-party library as follows:

```
namespace ex = std::execution;

// This returns a sender that does some piece of asynchronous work
// created by a third-party library, but parameterized with a custom
// allocator.
ex::sender auto make_async_work_with_alloc() {
    ex::sender auto work = third_party::make_async_work();

    return with_allocator(std::move(work), custom_allocator());
}
```

The sender returned by `third_party::make_async_work` might query for the allocator and use it to do allocations:

```
namespace third_party {
    namespace ex = std::execution;

    // A function that returns a sender that generates data on a special
    // execution context, populate a std::vector with it, and then completes
    // by sending the vector.
    constexpr auto _populate_data_vector =
        []<class Allocator>(Allocator alloc) {
            // Create an empty vector of ints that uses a specified allocator.

```

```

        using IntAlloc = std::allocator_traits<Allocator>::template
            rebind_alloc<int>;
        auto data = std::vector<int, IntAlloc>{IntAlloc{std::move(alloc)}};

        // Create some work that generates data and fills in the vector.
        auto work = ex::just(std::move(data))
            | ex::then([](auto data) {
                // Generate the data and fill in the vector:
                data.append_range(third_party::make_data())
                return data;
            });

        // Execute the work on a special third_party execution context:
        // (This uses the `on` as specified in P3175.)
        return ex::on(third_party_scheduler(), std::move(work));
    };

    // A function that returns the sender produced by
    // `_populate_data_vector`,
    // parameterized by an allocator read out of the receiver's environment.
    ex::sender auto make_async_work() {
        return ex::let_value(
            // This reads the allocator out of the receiver's execution
            // environment.
            ex::read_env(std::get_allocator),
            _populate_data_vector
        );
    }
}

```

§ 3.2 unstoppable

The unstoppable sender adaptor is a trivial application of `write_env` that modifies a sender so that it no longer responds to external stop requests. That can be of critical importance when the successful completion of a sender is necessary to ensure program correctness, *e.g.*, to restore an invariant.

The unstoppable adaptor might be implemented as follows:

```

inline constexpr struct unstoppable_t {
    template <sender Sndr>
    auto operator()(Sndr sndr) const {
        return write_env(std::move(sndr), prop(std::get_stop_token,
            never_stop_token()));
    }

    auto operator>() const {
        return write_env(prop(std::get_stop_token, never_stop_token()));
    }
} unstoppable {};

```

The section describing the finally adaptor will give a motivating example that makes use of unstoppable.

§ 3.3 finally

The C++ language lacks direct support for asynchronous destruction; that is, there is no way to say, “After this asynchronous operation, unconditionally run another asynchronous operation, regardless of how the first one completes.” Without this capability, there is no native way to have “async RAI”: the pairing the asynchronous acquisition of a resource with its asynchronous reclamation.

The finally sender adaptor captures the “async RAI” pattern in the sender domain. finally takes two senders. When connected and started, the finally sender connects and starts the first sender. When that sender completes, it saves the asynchronous result and then connects and starts the second sender. If the second sender completes successfully, the results from the first sender are propagated. Otherwise, the results from the second sender are propagated.

There is a sender in [exec] very much like finally as described above: schedule_from. The only meaningful difference is that in schedule_from, the “second sender” is always the result of calling schedule on a scheduler. With finally, the default implementation of schedule_from is trivial:

```
template <sender Sndr, scheduler Sched>
auto default-schedule-from-impl(Sndr sndr, Sched sched) {
    return finally(std::move(sndr), unstoppable(schedule(sched)));
}
```

This paper proposes repurposing the wording of schedule_from to specify finally, and then specifying schedule_from in terms of finally and unstoppable.

§ 3.3.1 Example: finally

In the following example, some asynchronous work must temporarily break a program invariant. It uses unstoppable and finally to restore the invariant.

```
namespace ex = std::execution;

ex::sender auto break_invariants(auto&... values);
ex::sender auto restore_invariants(auto&... values);

// This function returns a sender adaptor closure object. When applied
to
// a sender, it returns a new sender that breaks program invariants,
// munges the data, and restores the invariants.
auto safely_munge_data( ) {
    return ex::let_value( [](auto&... values) {
        return break_invariants(values...)
            | ex::then(do_munge) // the invariants will be restored even if
            `do_munge` throws
            | ex::finally(ex::unstoppable(restore_invariants(values...)));
    } );
}
```

```
auto sndr = ...;
scope.spawn( sndr | safely_munge_data() ); // See `counting_scope` from
P3149R2
```

§ 4 Discussion

There are a number of design considerations for the `finally` algorithm. The following questions have been brought up during LEWG design review:

§ 4.1 Should `finally` apply `unstoppable` by default to its second argument?

The observation was made that, since `finally` will often be used to do some cleanup operation or to restore an invariant, that operation should not respond to external stop requests, so `unstoppable` should be the default. It's a reasonable suggestion. Of course, there would need to be a way to override the default and allow the cleanup action to be canceled, and it isn't clear what the syntax for that would be. Another adaptor called `stoppable_finally`?

It is worth noting that `unstoppable` has uses besides `finally`, so it arguably should exist regardless of what the default behavior of `finally` is. Given that `unstoppable` should exist anyway, and that its behavior is pleasantly orthogonal to `finally`, the authors decided to keep them separate and let users combine them how they like.

§ 4.2 Is there a different design that better captures the “`async RAII`” intent?

Undoubtedly, the answer is “yes.” There are probably several such designs. One design that has been explored by Kirk Shoop is the so-called “`async object`” ([P2849R0]).

In Kirk's design, an `async object` has two basis operations: `async_construct` and `async_destruct`, both of which are asynchronous; that is, they all return senders. When `async_construct` completes, it does so with a handle to the object. The handle lets you interact with the object. Calling `async_destruct` on the object ends its lifetime.

A stop source, a run loop, and an `async scope` like [P3149R2]'s `counting_scope` can all be given the `async object` treatment. Multiple such `async resources` can be used in tandem, as in the following example:

```
namespace ex = std::execution;

// In this example, stop_source_object, run_loop_object, and
// counting_scope_object
// all satisfy the async_object concept.
stop_source_object stop;
run_loop_object loop;
counting_scope_object scope;
```

```

auto thread_fn = [] (auto stop, auto loop, auto scope) {
    // `thread_fn` must return a sender that uses the handles passed in.
    packaged_async_object thread{thread_object{}, [loop]{ loop.run(); }};

    auto work_fn = [=] (auto /*thread*/) {
        // construct a stop callback that requests stop on the run_loop when
        // stop is requested on the stop_source_object.
        auto stop_callback = [] <class Token, class Fn> (Token tok, Fn fn) {
            return stop_callback_of_t<Token, Fn>{tok, fn};
        } (stop.get_token(), [loop] { loop.request_stop(); });

        // spawn 1000 tasks on the run_loop in the counting_scope.
        for (int i = 0; i < 1000; ++i) {
            auto work = ex::then(ex::just(), [=]{ do_work(i); });
            ex::spawn(ex::on(loop.get_scheduler(), std::move(work)), scope);
        }

        // Tell the run_loop to stop when it has finished processing the work:
        scope_guard guard{[] { loop.request_stop(); }};
        return ex::just();
    };

    return async_using(work_fn, thread);
};

// This sender, when connected and started, will async-construct the
// async objects, passes them as arguments to the thread_fn, start
// the returned sender, and when it completes, async-destruct the
// async objects in the reverse order of their construction:
ex::sender auto use = async_using(thread_fn, stop, loop, scope);

// Launch it all and wait for it to complete:
std::this_thread::sync_wait(std::move(use));

```

This design nicely captures the “async RAII” pattern. A type modeling the async object concept is like an async class with an async constructor and an async destructor. Instead of using `finally`, a user can implement a type that satisfies the `async_object` concept.

Although there are times when it is appropriate to model the `async_object` concept as presented in [P2849R0], doing so is certainly more work than just using `finally`. One can think of `finally` as an *ad hoc* form of async RAII. To draw an analogy, `finally` is to `async_object` what `scope_guard` is to custom RAII wrappers like `unique_ptr`. That is no diss on `scope_guard`; it has its place!

So too does `finally` in the authors’ opinion. It captures a common pattern quite simply, and is not a far departure from what is in [exec] already. An async object abstraction is a much heavier lift from a standardization point of view. Pursuing that design instead of `finally` risks missing the C++26 boat, leaving users without a standard way to reliably clean up asynchronous resources.

In the end, the authors expect that we will have both, just as many codebases make use of both `scope_guard` and `unique_ptr`.

§ 5 Proposed Wording

[Editor's note: The wording in this section is based on the current working draft.]

[Editor's note: Change [exec.syn] as follows:]

```
inline constexpr unspecified write_env{};  
inline constexpr unspecified unstoppable{};  
inline constexpr start_on_t start_on{};  
inline constexpr continue_on_t continue_on{};  
inline constexpr on_t on{};  
inline constexpr unspecified finally{};  
inline constexpr schedule_from_t schedule_from{};
```

[Editor's note: Replace all instances of “*write-env*” with “*write_env*”. After [exec.adapt.objects], add a new subsection “*execution::write_env* [exec.write.env]” and move the specification of the exposition-only *write-env* from [exec.snd.general]/p3.15 into it with the following modifications:]

(34.9.11.?)

execution::write_env [exec.write.env]

1. ~~*write-env*~~*write_env* is ~~an exposition-only~~a sender adaptor that accepts a sender and a queryable object, and that returns a sender that, when connected with a receiver *rcvr*, connects the adapted sender with a receiver whose execution environment is the result of joining the queryable ~~argument-env~~object to the result of *get_env(rcvr)*.
2. Let *write-env-t* be an exposition-only empty class type.
3. *Returns:* *make-sender(make-env-t(), std::forward<Env>(env), std::forward<Sndr>(sndr)).*
2. *write_env* is a customization point object. For some subexpressions *sndr* and *env*, if *decltype(sndr)* does not satisfy sender or if *decltype(env)* does not satisfy *queryable*, the expression *write_env(sndr, env)* is ill-formed. Otherwise, it is expression-equivalent to *make-sender(write_env, env, sndr)*.
3. ~~*Remarks:*~~ The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for ~~*write-env-t*~~*write_env* as follows:

```
template<>  
struct impls-for<write-env-tdecayed-typeof<write_env>> : default-impls {  
    static constexpr auto get-env =  
        [](auto, const auto& env, const auto& rcvr) noexcept {  
            return JOIN-ENV(env, get_env(rcvr));  
        };  
};
```


[Editor's note: After [exec.write.env], add a new subsection “execution::unstoppable [exec.unstoppable]” as follows:]

(34.9.11.?) **execution::unstoppable [exec.unstoppable]**

1. `unstoppable` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver but with a `never_stop_token` as the value of the `get_stop_token` query.
2. For a subexpression `sndr`, `unstoppable(sndr)` is expression equivalent to `write_env(sndr, MAKE_ENV(get_stop_token, never_stop_token{}))`.

[Editor's note: Change subsection “execution::schedule_from [exec.schedule.from]” to “execution::finally [exec.finally]”, change every instance of “schedule_from” to “finally” and “schedule_from_t” to “*decayed-typeof*<finally>”, and change the subsection as follows:]

(34.9.11.5) **execution::finally [exec.finally]**

[Editor's note: Replace paragraphs 1-3 with the following:]

1. `finally` is a sender adaptor that starts one sender unconditionally after another sender completes. If the second sender completes successfully, the `finally` sender completes with the async results of the first sender. If the second sender completes with error or stopped, the async results of the first sender are discarded, and the `finally` sender completes with the async results of the second sender. [*Note*: It is similar in spirit to the `try/finally` control structure of some languages. — *end note*]
2. The name `finally` denotes a customization point object. For some subexpressions `try_sndr` and `finally_sndr`, if `try_sndr` or `finally_sndr` do not satisfy sender, the expression `finally(try_sndr, finally_sndr)` is ill-formed.
3. Otherwise, the expression `finally(try_sndr, finally_sndr)` is expression-equivalent to `make_sender(finally, {}, try_sndr, finally_sndr)`.
4. Let `CS` be a specialization of `completion_signatures` whose template parameters are the pack `Sigs`. Let `VALID-FINALLY(CS)` be true if and only if there is no type in `Sigs` of the form `set_value_t(Ts...)` for which `sizeof...(Ts)` is greater than 0. Let `F` be `decltype(finally_sndr)`. If `sender_in<F>` is true and `VALID-FINALLY(completion_signatures_of_t<F>)` is false, the program is ill-formed.
5. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `finally` as follows:

```
namespace std::execution {  
    template<>
```

```

struct impls-for<decayed-typeof<finally>> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
};
}

```

1. The member `impls-for<decayed-typeof<finally>>::get-attrs` is initialized with a callable object equivalent to the following lambda:

```

[](const auto& data, const auto& child) noexcept ->
decltype(auto) {
    return JOIN-ENV(SCHED-ATTRS(data), FWD-
ENV(get_env(child)));
}

```

```

[](auto, const auto& tsndr, const auto& fsndr)
noexcept -> decltype(auto) {
    return JOIN-ENV(FWD-ENV(get_env(fsndr)), FWD-
ENV(get_env(tsndr)));
}

```

2. The member `impls-for<decayed-typeof<finally>>::get-state` is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
noexcept(see below)
requires sender_in<child-type<Sndr, 0>, env_of_t<Rcvr>>
&&
sender_in<child-type<Sndr, 1>, env_of_t<Rcvr>> &&
VALID-FINALLY(completion signatures of t<child-
type<Sndr, 1>, env_of_t<Rcvr>>) {

    auto& [_, sch, child_, tsndr, fsndr] = sndr;

using sched_t = decltype(auto(sch));
using fsndr_t = decltype(std::forward like<Sndr>
(fsndr));
    using variant_t = see below;
    using receiver_t = see below;

    using operation_t =
    connect_result_t<schedule_result_t<sched_t>fsndr_t,
    receiver_t>;
    constexpr bool nothrow =
    noexcept(connect(schedule(sch)std::forward like<Sndr>
(fsndr), receiver_t{nullptr}));

    struct state-type {
        Rcvr& rcvr; // exposition only
        variant_t async-result; // exposition only
        operation_t op-state; // exposition only

        explicit state-type(sched_t schfsndr_t fsndr, Rcvr&
rcvr) noexcept(nothrow)
            : rcvr(rcvr), op-
state(connect(schedule(sch)std::forward like<Sndr>
(fsndr), receiver_t{this})) {}
    }
}

```

```
};

    return state-type{schstd::forward like<Sndr>(fsndr),
        rcvr};
}
```

1. Objects of the local class *state-type* can be used to initialize a structured binding.
2. Let *Sigs* be a pack of the arguments to the *completion_signatures* specialization named by *completion_signatures_of_t<child-type<Sndr, _____0>, env_of_t<Rcvr>>*. Let *as-tuple* be an alias template that transforms a completion signature *Tag(Args...)* into the tuple specialization *decayed-tuple<Tag, Args...>*. Then *variant_t* denotes the type *variant<monostate, as-tuple<Sigs>...>*, except with duplicate types removed.
3. *receiver_t* is an alias for the following exposition-only class:

```
namespace std::execution {
    struct receiver-type {
        using receiver_concept = receiver_t;
        state-type* state; // exposition only

        void set_value() && noexcept {
            visit(
                [this]<class Tuple>(Tuple& result) noexcept -
                > void {
                    if constexpr (!same_as<monostate, Tuple>) {
                        auto& [tag, ...args] = result;
                        tag(std::move(state->rcvr),
                            std::move(args)...);
                    }
                },
                state->async-result);
        }

        template<class Error>
        void set_error(Error&& err) && noexcept {
            execution::set_error(std::move(state->rcvr),
                std::forward<Error>(err));
        }

        void set_stopped() && noexcept {
            execution::set_stopped(std::move(state->rcvr));
        }

        decltype(auto) get_env() const noexcept {
            return FWD-ENV(execution::get_env(state-
                >rcvr));
        }
    };
}
```

4. The expression in the `noexcept` clause of the lambda is true if the construction of the returned *state-type* object is not potentially throwing; otherwise, false.
3. The member *impls-for<decayed-typeof<finally>>::complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class Tag, class... Args>(auto, auto& state, auto& rcvr,
    Tag, Args&&... args) noexcept -> void {
    using result_t = decayed-tuple<Tag, Args...>;
    constexpr      bool      nothrow      =
    is_nothrow_constructible_v<result_t,      Tag,
    Args...>;

    TRY-EVAL(std::move(rcvr), [&]() noexcept(nothrow) {
        state.async-result.template emplace<result_t>(Tag(),
            std::forward<Args>(args)...);
    }());

    if (state.async-result.valueless_by_exception())
        return;
    if (state.async-result.index() == 0)
        return;

    start(state.op-state);
};
```

[Editor's note: Remove paragraph 5, which is about the requirements on customizations of the algorithm; *finally* cannot be customized.]

[Editor's note: Insert a new subsection “`execution::schedule_from` [exec.schedule.from]” as follows:]

(34.9.11.?) **`execution::schedule_from` [exec.schedule.from]**

[Editor's note: These three paragraphs are taken unchanged from [exec].]

1. `schedule_from` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource. [*Note*: `schedule_from` is not meant to be used in user code; it is used in the implementation of `continue_on`. — *end note*]
2. The name `schedule_from` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy scheduler, or `Sndr` does not satisfy sender, `schedule_from` is ill-formed.
3. Otherwise, the expression `schedule_from(sch, sndr)` is expression-equivalent to:

```
transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(schedule_from, sch, sndr));
```

except that `sch` is evaluated only once.

4. The exposition-only class template *impls-for* is specialized for `schedule_from_t` as follows:

```
template<>
struct impls-for<schedule_from_t> : default-impls {
    static constexpr auto get-attrs =
        [](const auto& data, const auto& child) noexcept ->
        decltype(auto) {
            return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
        };
};
```

5. Let `snr` and `env` be subexpressions and let `Snr` be `decltype((snr))`. If *sender-for*<`Snr`, `schedule_from_t`> is false, then the expression `schedule_from.transform_sender(snr, env)` is ill-formed; otherwise, it is equal to:

```
auto&& [tag, sch, child] = snr;
return finally(std::forward_like<Snr>(child),
               unstopable(schedule(std::forward_like<Snr>(sch))));
```

[*Note:* This causes the `schedule_from(sch, snr)` sender to become `finally(snr, unstopable(schedule(sch)))` when it is connected to a receiver with an execution domain that does not customize `schedule_from`. — *end note*]

[Editor's note: The following paragraph is taken unchanged from [exec].]

6. Let `out_snr` be a subexpression denoting a sender returned from `schedule_from(sch, snr)` or one equal to such, and let `OutSnr` be the type `decltype((out_snr))`. Let `out_rcvr` be a subexpression denoting a receiver that has an environment of type `Env` such that `sender_in<OutSnr, Env>` is true. Let `op` be an lvalue referring to the operation state that results from connecting `out_snr` with `out_rcvr`. Calling `start(op)` shall start `snr` on the current execution agent and execute completion operations on `out_rcvr` on an execution agent of the execution resource associated with `sch`. If scheduling onto `sch` fails, an error completion on `out_rcvr` shall be executed on an unspecified execution agent.

[Editor's note: The following changes to the `let_*` algorithms are not strictly necessary; they are simplifications made possible by the addition of the `write_env` adaptor above.]

[Editor's note: Remove [exec.let]p5.1, which defines an exposition-only class *receiver2*.]

[Editor's note: Change [exec.let]p5.2.2 as follows:]

2. Given a type `Tag` and a pack `Args`, let *as-snr2* be an alias template such that *as-snr2*<`Tag`(`Args...`)> denotes the type *call-result-t<decayed-*

`typeof<write_env>, call_result_t<Fn, decay_t<Args>&...>, Env>`. Then `ops2_variant_t` denotes the type `variant<monostate, connect_result_t<as_sndr2<LetSigs>, receiver2<Rcvr, Env>>...>`.

[Editor's note: Change [exec.let]p5.3 as follows:]

3. The exposition-only function template *let-bind* has effects equivalent to:

```
using args_t = decayed_tuple<Args...>;
auto mkop2 = [&] {
    return connect(
        write_env(
            apply(std::move(state.fn),
                state.args.template emplace<args_t>(std::forward<Args>
                    (args)...)),
            std::move(state.env)),
        receiver2{rcvr, std::move(state.env)}std::move(rcvr));
};
start(state.ops2.template emplace<decltype(mkop2())>(emplace-
    from{mkop2}));
```

§ 6 References

[P2849R0] Kirk Shoop. 2024-05-21. `async-object` - aka `async-RAII` objects.

<https://wg21.link/p2849r0>

[P3149R2] Ian Petersen, Ján Ondrušek; Jessica Wong; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; 2024-03-20. `async_scope` — Creating scopes for non-sequential concurrency.

<https://wg21.link/p3149r2>

[P3175R0] Eric Niebler. 2024-03-14. Reconsidering the `std::execution::on` algorithm.

<https://wg21.link/p3175r0>